

Teaching a Bank to Score Risk It Has Never Seen: A Credit Model Built on Shopping Behavior, Not Credit History

How Bati Bank's buy-now-pay-later partnership turns eCommerce transaction data into a real-time credit risk API — and why the target variable itself is the most important design decision in the whole project.

The problem: a credit model with no default label

Bati Bank is partnering with an eCommerce platform to offer buy-now-pay-later (BNPL) credit. The bank's job is the same one it's always had — decide who gets credit, and how much — but the data is unfamiliar. Instead of a loan book with repayment history, Bati Bank has something else entirely: 95,662 raw transaction records from the Xente platform, covering 3,742 customers over a 90-day window (mid-November 2018 to mid-February 2019). Every row shows what a customer bought, through which channel, for how much, and when. None of it says whether that customer ever failed to repay a loan — because none of these customers has taken a loan yet. The BNPL product doesn't exist for them yet either.

This is the central fact that shapes every decision in this project: **there is no default label to learn from.** Before any model gets trained, the team has to answer a harder question first — what does “risk” even mean here, and how do you measure it from shopping behavior alone?

Why this isn't just a machine learning problem

The Basel II Capital Accord is the reason a bank can't just throw the data at a gradient boosting model and ship whatever comes out. Basel II's emphasis on risk measurement means a credit model has to be *explainable*, not just accurate — the bank needs to be able to say why a customer was scored the way they were, document every assumption behind the target variable, and monitor the model's behavior over time. A black-box score with no audit trail is not defensible in a regulated lending context, no matter how good its ROC-AUC looks on a held-out test set.

That requirement runs through the whole build:

- The target variable had to be constructed, not found — and that construction has to be documented and defensible, not just “whatever K-Means produced.”
- The feature pipeline had to be deterministic and reproducible (fixed random seeds everywhere: clustering, train/test splits, hyperparameter search).
- Model comparison had to include an interpretable baseline (Logistic Regression), not just the best-performing model.

- Every experiment had to be tracked — parameters, metrics, and artifacts — so the choice of final model has a paper trail.

Building the proxy target: RFM segmentation as a stand-in for default

Since there's no default label, the project builds a **proxy**: a behavioral signal that stands in for credit risk until real repayment data exists. The method is RFM analysis — Recency, Frequency, Monetary value — a classic customer-segmentation technique repurposed here as a risk signal.

For each customer:

- **Recency**: days since their last transaction, measured from a snapshot date one day after the last transaction in the dataset (2019-02-14).
- **Frequency**: total number of transactions.
- **Monetary**: total transaction value.

These three numbers were scaled and fed into K-Means with $k=3$ and a fixed `random_state=42` for reproducibility. The intent: separate customers into an actively-engaged segment, a moderately-engaged segment, and a disengaged segment, then treat the disengaged segment as the high-risk proxy — the assumption being that a customer who has stopped transacting, transacts rarely, and spends little is a customer the bank knows the least about and should be most cautious with.

The first version of this clustering was wrong, and it's worth explaining why. Monetary value in this dataset is extremely skewed — a small number of customers each generated tens of millions of shillings in transaction value, dwarfing the typical customer's total. Clustering on the raw scaled values let those few outliers dominate the distance metric: K-Means produced one cluster of 4 extreme-value customers and lumped everyone else into two much larger, less meaningful groups. The fix was a `log1p` transform on Frequency and Monetary before scaling — standard practice for heavy-tailed variables, and one the Task 1 EDA had already flagged as necessary. After the fix, K-Means produced three genuinely balanced segments (1,173 / 1,202 / 1,367 customers), and the cluster boundaries lined up with intuition: the smallest, most disengaged group had a mean recency of 62 days versus 13 days for the most engaged group.

The high-risk cluster itself isn't picked by eyeballing the numbers — it's selected automatically, by z-scoring each cluster's mean Recency, Frequency, and Monetary value and picking the cluster with the worst composite score (highest recency, lowest frequency, lowest monetary). That cluster became `is_high_risk = 1` for **1,173 customers (31.3% of the base)**.

The honest caveat

This proxy is a modeling assumption, not measured truth, and it carries real business risk:

- **Proxy risk**: the model learns “disengagement,” not “default.” A customer who stopped shopping on this platform because they switched to a competitor is not the same as a customer who would fail to repay a loan.

- **Exclusion risk:** a low-activity but genuinely creditworthy customer could be scored as high-risk and denied credit or offered worse terms, purely because they don't shop often — not because they're a bad credit risk.
- **Feedback-loop risk:** once the bank starts approving or denying credit based on this model, it stops observing outcomes for the customers it denies. Future retraining data will be biased toward the customers the model already approved.
- **Product-launch risk:** behavior on the platform *before* BNPL exists may not represent behavior *after* customers know credit is on offer.

None of this is a reason not to build the model — it's a reason to treat it as a first-generation decision-support signal, not an unchecked auto-approval engine, and to revisit the proxy the moment real repayment data exists.

From transactions to features: an engineering decision that matters more than it looks

Credit decisions happen per customer, but the raw data is per transaction. The feature pipeline (`src/data_processing.py`, expressed as a single `sklearn.Pipeline`) rolls 95,662 transaction rows up into 3,742 customer-level records, computing things like total/average/median transaction amount, transaction count, debit/credit split, category and channel diversity, active days, and time-of-day patterns — 45 columns in total after encoding and scaling.

One design decision is worth calling out explicitly, because it's easy to get wrong: **the classifier is never trained on Recency, Frequency, or Monetary directly**. Those three numbers are exactly what the K-Means proxy label was built from — a correlation check confirmed Recency alone correlates with `is_high_risk` at **0.85**. Feeding them into the classifier wouldn't build a predictive model; it would just teach the model to reverse-engineer which side of the clustering boundary a customer fell on, which is both circular and useless in production, since a brand-new applicant has no comparable RFM history to re-cluster against anyway. The classifier instead learns from the *other* engineered features — transaction count, active months, category and channel diversity, and so on — which still correlate meaningfully with the proxy label (0.2–0.4 range) without being definitionally identical to it.

Comparing two models, honestly

Two classifiers were trained and tuned on a stratified 80/20 train/test split (`random_state=42` throughout):

Model	Tuning	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	GridSearchCV	70.2%	52.4%	56.2%	54.2%	0.800
XGBoost (best)	RandomizedSearchCV	75.7%	61.8%	59.1%	60.4%	0.851

Every run — parameters, metrics, and the model artifact itself — was logged to MLflow. XGBoost wins on every metric and was registered as `credit-risk-is-high-risk` (version 1) in the MLflow Model Registry.

That said, the interpretability-versus-performance trade-off from the Task 1 business understanding is real here, not theoretical. Logistic Regression's coefficients can be read directly by a risk analyst; XGBoost's decision boundary can't be inspected the same way without additional tooling (SHAP values, partial dependence plots) that this phase of the project didn't build out. XGBoost's ~5-point ROC-AUC advantage is meaningful, but a bank moving this into production would need to weigh that gain against the added governance cost of explaining a boosted-tree model's decisions to a credit committee — exactly the trade-off Basel II's documentation expectations are designed to force into the open, rather than letting the highest leaderboard number win by default.

Shipping it: an API that takes raw transactions, not pre-computed features

The registered MLflow model isn't just the XGBoost classifier — it's the *entire* serving pipeline: the fitted feature-engineering steps plus the trained classifier, bundled into one object. This matters operationally. It means the FastAPI service (`src/api/main.py`) can accept the same raw transaction data the eCommerce platform already produces, and the model handles feature engineering internally — no risk of the API's feature computation drifting out of sync with what the model was actually trained on.

A real request against the running service, scoring two customers from held-out transaction history:

```
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"transactions": [ ...raw transaction rows for CustomerId_4406 and CustomerId_988... ]}'

{
  "predictions": [
    {"CustomerId": "CustomerId_4406", "risk_probability": 0.000770587648730725},
    {"CustomerId": "CustomerId_988", "risk_probability": 0.0004339662555139512}
  ]
}
```

Both customers here score very low risk — consistent with them being active, frequent transactors in the underlying data. The `/predict` endpoint accepts multiple transactions per customer (required, since a single transaction can't capture the behavioral pattern the model was trained on) and returns one probability per unique CustomerId. A `/health` endpoint reports service status independently of whether a model is currently loaded, so the container can be monitored even if model loading fails — in which case `/predict` returns a clear 503 rather than the process crashing.

The whole thing is packaged behind a Dockerfile and `docker-compose.yml`, with a GitHub Actions workflow (`.github/workflows/ci.yml`) that runs `flake8` and the full `pytest` suite — 16 tests spanning feature engineering, RFM clustering and reproducibility, training helpers, and the API itself (using a mocked model, so CI doesn't need a trained model artifact to verify the service's request/response contract) — on every push and pull request against main.

What this project is, and isn't

This is a working, end-to-end credit risk scoring service: real data in, a documented and reproducible proxy target, two compared and tracked models, and a deployable API that returns a risk probability for a

real customer in real time. That's a genuine capability Bati Bank didn't have before.

What it isn't, yet, is a validated default model. The `is_high_risk` label is a considered assumption about what disengagement implies about creditworthiness, not a measurement of it. The honest next step — the one this report keeps returning to — is closing the loop: once the BNPL product launches and real repayment outcomes start accumulating, those outcomes need to replace the RFM proxy as the training target, and the model built here should be judged against how well its original assumption held up, not treated as a finished product.

Code, tests, and full experiment history: github.com/maedotamha/Credit-Risk-Probability-Model-